Reproducibility Failures in Quantized Model Evaluation: Item Ordering Moves 4-Bit Accuracy and Leaves `bf16` Exactly Unchanged

Joseph Sankoorikal Johny
Independent Researcher
Seattle, USA
joseph.sankoorikal@gmail.com

Syed Ahmed Danish
Independent Researcher
Hartford, USA
syed.danish47@gmail.com

Aniruddha Sharma
Independent Researcher
Seattle, USA
anirudh217sharma@gmail.com

***Abstract*—A reported quantization cost is one score minus another, so any part of the evaluation that penalises the two models unequally ends up inside the reported gap. We evaluate three instruction-tuned models at five precisions with the engine, decoding and prompts fixed, and vary properties of the evaluation that protocols do not report. Two results are established by intervention or by a pre-registered contrast that survives multiple-comparison correction. First, item submission order changes 4-bit accuracy by up to 1.09 points of standard deviation across 10 orderings and leaves `bf16` *exactly* unchanged, with the engine's batch-invariant mode enabled: that mode does not cover the quantized matrix-multiply kernels, and disabling them removes the variation for GPTQ and AWQ. The GPTQ result replicates on a second GPU architecture. Second, the extraction rule changes the measured quantization effect with a task-dependent sign, moving GPTQ W4A16 relative to `bf16` by +1.73 points on BBH and by -5.21 on GSM8K, so no single correction restores comparability. Supporting results bound the picture, and we give remedies with measured costs, a released corpus of 105,950 generations and a runnable order-invariance probe.**

***Index Terms*—quantization, reproducibility, LLM evaluation, inference kernels, batch invariance**

I. Introduction

When a paper reports that 4-bit quantization costs two points of accuracy, that number was not measured. It is the difference between two measurements, and anything about the evaluation that penalises the two runs unequally lands in the reported gap. Kurtic et al. [1] showed that reported degradation had been overstated and traced the excess to untuned quantization-side hyperparameters. Our concern is the residual that survives that tuning, and our mechanisms act on the evaluation, not on the quantized model.

Changing every score is not the same as changing the gap between scores. Lenient answer extraction is known to raise absolute scores [2, 3], and prompt formatting moves them substantially [4]. A choice that lowers every model's score equally is a nuisance, because the gap survives it. A choice that lowers some models' scores more than others changes the gap, and so contaminates every published comparison that did not report it. We test for the second kind, and find it.

Two headline results. Varying only the order in which the same items are submitted changes 4-bit accuracy by up to 1.09 points of standard deviation and leaves `bf16` exactly unchanged, because the engine's batch-invariant mode does not cover the quantized kernels it selects by default; disabling them removes the variation (§IV-A). And the extraction rule interacts with precision with a sign that depends on the task, so no single correction recovers comparability (§IV-B).

This is not an argument for scoring more permissively. Ren et al. [5] find that 8.2–44.1% of answers graded correct are reached by shortcut hacking, and we find that lenient extraction also destroys some correct answers. The goal is to *calibrate* extraction, not to loosen it. Nor is it an argument that quantization is benign: benchmarks also *understate* quantization damage relative to human judgement [6, 7]. Both directions hold at once, because they are different quantities.

Contributions. (i) Item ordering moves 4-bit accuracy and leaves `bf16` exactly unchanged, with the mechanism pinned by intervention to kernels the batch-invariant mode does not cover, and replicated on a second GPU architecture. (ii) The precision-by-scoring interaction changes sign across tasks and differs across methods. (iii) Supporting results: a cross-process reproducibility failure, a bounded null for the generation budget, and a log-likelihood control on ARC. (iv) Remedies with measured costs, and three fields every protocol should report. (v) A released corpus of 105,950 generations and an order-invariance probe.

II. Related Work

Pape et al. [8] is the nearest prior work. They survey 35,000 publications, find that only 36% of papers without released code name even their inference backend, and show that switching backend alone moves GSM8K by 16.6 points; Masoudian et al. [9] attribute roughly 39% of out-of-the-box variance to the serving configuration. But their controlled comparison runs at *batch size one*, chosen to prevent batching-induced instability, at FP16. The instability they design out is the one we study: we fix the engine, weights and decoding, vary only

which items share a batch, and attribute the divergence to a named quantized kernel.

He and Thinking Machines Lab [10] traced nondeterministic inference to reductions whose order depends on how many requests share a batch, and introduced the batch-invariant kernels that vLLM's batch-invariant mode builds on. We show that its guarantee does not extend to the 4-bit GPTQ and AWQ kernels vLLM selects by default. Hua et al. [11] ask whether prompt sensitivity is a model property or an evaluation artifact; we vary the scorer rather than the prompt. Whether quantization lengthens chain-of-thought is disputed [12, 13, 14], and no study on either side sweeps the generation cap. Kurt [15] report strict-versus-flexible GSM8K gaps of 41 to 64 points across `llama.cpp` quantizations without interpreting them.

III. Method

Design. Three instruction-tuned models (Qwen2.5-3B, Phi-3.5-mini, Llama-3.1-8B), each pinned by commit, at five precisions: `bf16`, GPTQ-8bit, GPTQ-4bit, AWQ-4bit and bitsandbytes NF4. Tasks are GSM8K with chain-of-thought, BIG-Bench Hard and ARC-Challenge, evaluated with the lm-evaluation-harness [16]. Every manipulation is one factor at a time from a single fixed reference condition.

Quantization. GPTQ and AWQ are produced by one tool at one normalised calibration budget (512 sequences of up to 2048 tokens from `ultrachat_200k`), identical for every arm and model, because the tool's own examples differ by $8\times$ in budget between the two methods. Group size is 128, and activation reordering is explicitly disabled to match the vendors' released checkpoints. NF4 is quantized in flight at load.

Scoring. Every generation is stored once and scored by a ladder of extractors, or *rungs*, on the same text, so the ladder costs no extra generation: *strict*, the harness's canonical regular expression; *strict-last*, the same expression taking the last match; *permissive*, the harness's flexible extractor; *symbolic* equivalence; and a trained judge [17] that we did not validate and on which no claim rests. The headline contrast is strict against permissive, both deterministic and auditable. We validated both against 250 human labels from two annotators (raw agreement 0.975, Gwet's AC1 0.968). Auditing our own pipeline found that a rule we had added ahead of the harness's returned worse answers on 2,068 of 105,950 generations, which our unit tests never caught. Repairing it changed the conclusion of two interaction estimates and the sign of one; all numbers here are after the repair.

Compliance and truncation. Every generation records the engine's termination reason and its token count, which cannot be reconstructed later, so a wrong answer can be split into one the model did not know, one it knew but did not state in the expected form, and one cut off by the budget. We also separate an answer being *extractable* from the model having *committed* to it, because a fallback that takes the last number in a truncated trace extracts something the model never asserted.

Serving. One engine (vLLM 0.26.0), greedy decoding, every sampling field pinned, batch-invariant mode on [10], tensor parallelism of one, prefix caching off, and a maximum of 32 co-resident sequences. That value was measured rather than assumed: it is the largest at which every arm is bitwise reproducible across repeated identical runs, and any arm less reproducible than `bf16` sets a noise floor below which we claim no effect. The full configuration, rubric and pre-registration are in the supplementary material.

TABLE I
Accuracy across 10 orderings of the same 500 items (Qwen2.5-3B, L40S, $B = 32$), in accuracy points. The standard deviation is the inferential quantity; the range grows with the number of orderings.

	strict		permissive	
arm	SD	range	SD	range
BF16	0.00	0.00	0.00	0.00
GPTQ W4A16	0.87	2.80	0.75	2.40
AWQ W4A16	0.52	2.00	1.09	4.20
bnb NF4	0.11	0.40	0.10	0.20

IV. Results

The first two subsections are the headline results, established by intervention (§IV-A) or by a pre-registered contrast that survives Holm–Bonferroni (§IV-B). The third collects supporting results.

A. Item ordering moves 4-bit accuracy and leaves `bf16` unchanged

Requests are batched dynamically, so an item's position decides which other items share its batch, and so the order in which the engine sums intermediate results. Floating-point addition is not associative, and in chain-of-thought generation one differing token changes every token after it. Running the same items again in the *same* order changes nothing: none of the 15 model-and-precision combinations produced a single differing output. Every difference below is caused by the ordering.

Accuracy moves, and only for quantized arms. Under all 10 orderings, `bf16` accuracy is identical at both rungs (Table I). Every quantized arm moves, by up to 1.09 points of standard deviation and a range of up to 4.2. The shift is deterministic given an ordering, so repeated runs cannot average it away, and it moves accuracy in both directions, so no ordering is the correct one. We report the standard deviation rather than the spread because the spread grows with effort: the same quantity over three orderings gave a spread of 1.6 points, and over 10 gives 4.2, with no change to the underlying effect.

The batch-invariant mode does not cover Marlin or Machete. Disabling those two kernels through `VLLM_DISABLED_KERNELS`, and changing nothing else, drives divergence to zero for GPTQ and AWQ, on text and on accuracy (Table II). Both of our checkpoints use the `compressed-tensors` format, which vLLM loads through a 4-bit kernel chooser with no batch-invariance check, in 0.26.0 and in the source of 0.32.0, the newest release in

TABLE II
PAIRED INTERVENTION AT $B = 8$: THE SAME INPUTS AND ENGINE WITH AND WITHOUT THE MARLIN AND MACHETE KERNELS. TEXT DIVERGENCE IN PERCENT, ACCURACY SD IN POINTS.

	text divergence		accuracy SD	
arm	Marlin on	off	Marlin on	off
GPTQ W4A16	53.2	0.0	0.42	0.00
AWQ W4A16	53.6	0.0	0.35	0.00
bnb NF4	49.4	49.4	0.31	0.31

October 2026; it selects Marlin whether or not the mode is on. The source shows four such gaps: that chooser, the AutoGPTQ loader, the sampler, and kernel autotuning. The only guard we found, which avoids Marlin in this mode, is in the loader for AutoAWQ-format checkpoints, a path ours never take. NF4 never used these kernels and is unchanged by the intervention, so its ordering sensitivity has a second, separate cause. The intervention locates the mechanism; it does not explain the arithmetic, and we do not identify which reduction differs.

The effect replicates on a second GPU architecture. On an NVIDIA A10G (Ampere, SM 8.6) with the same image, engine, weights and probe, `bf16` text differs on 0.0% of items across 10 orderings. GPTQ W4A16 text differs on 69.8% (71.2% on the L40S), with a strict accuracy SD of 0.70 (0.87 on the L40S), and disabling Marlin and Machete takes it from 53.4% to 0.0%.

B. The scoring rule changes the measured quantization effect, unevenly

For each arm we take the strict-to-permissive gain in accuracy and subtract `bf16`'s gain on the same items, bootstrapped over item clusters. Positive means leniency helps the quantized arm more than `bf16`.

The interaction is real, and its sign depends on the task. 5 of the 8 BBH and GSM8K estimates survive Holm–Bonferroni (Table III). On BBH, leniency helps the 4-bit arms more than `bf16`: GPTQ W4A16 +1.73 pp and AWQ W4A16 +1.55 pp. On $\text{GSM8K}_{\text{CoT}}$ it reverses: GPTQ W4A16 -5.21 pp and NF4 -8.44 pp.

The disagreement is tested, not observed. Two contrasts were pre-registered and named structurally before either was computed. Holding the arm fixed and varying the task gives +6.93 pp $[+5.09, +8.76]$; holding the task fixed and comparing GPTQ with AWQ at matched bit width gives -4.30 pp $[-5.84, -2.75]$. Both reject under Holm–Bonferroni. A uniform interaction would have a remedy, one offset. A heterogeneous one does not: the correction that fits GPTQ on BBH has the wrong sign on GSM8K, so the extraction rule has to be reported.

GPTQ W8A16 is null on both generative tasks, as a dose-response predicts, and ARC-C is exactly zero for every arm, bounded within ± 0.51 pp at n=1,500. GSM8K under the hash-marker prompt is excluded from the family, because

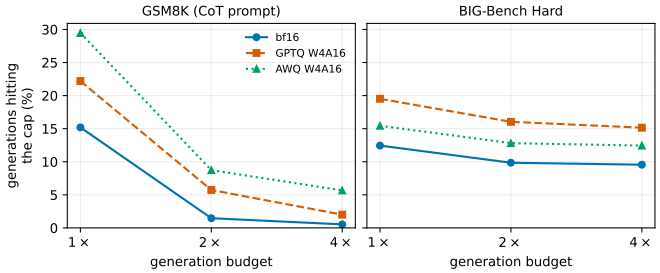


Fig. 1. Generations stopped by the budget rather than by the model, at three budgets.

instruction-tuned models rarely emit the `####` marker unprompted and strict extraction there measures marker compliance; its estimates are large and negative (GPTQ W4A16 -13.12 pp, AWQ W4A16 -7.91 pp), and we show them only as the extreme of that gradient.

C. Supporting results

Reproducibility holds within a process and fails across processes. Within one process, generation is bitwise reproducible: zero differing items across 42,380 paired comparisons. Across processes, the same configuration diverges on up to 45% of items for one model. A practitioner who repeats a measurement in one process therefore concludes the pipeline is deterministic, using exactly the check a careful person would run. We refuted four causes (KV-cache sizing, memory-driven batching, weight provenance and sampling defaults) and do not identify the real one.

A fixed generation budget truncates quantized outputs more often, at a bounded accuracy cost. We ran 12,600 items at the default budget and at two and four times it (Fig. 1). On GSM8K, raising the budget cuts the share of generations stopped by the cap from 15.2% to 0.5% for `bf16`, from 22.2% to 2.0% for GPTQ and from 29.5% to 5.7% for AWQ. But no change in measured degradation survives Holm–Bonferroni: the smallest p is 0.048 against a threshold of 0.0125. We report this as a bound, not an effect. The mechanism is nonetheless visible in tokens, and it settles the dispute of §II: on GSM8K the mean scored answer is 169 tokens for `bf16`, 173 for GPTQ and 180 for AWQ, and on BBH at the largest budget GPTQ writes 415 tokens past its scored answer against 215 for `bf16`.

Quantization shifts whether the model answers at all. Against `bf16` on the same items, 48 of 108 per-cell comparisons change commitment significantly, in both directions. For GPTQ W4A16 on BBH, 423 items lose a committed answer and 421 gain one, a net of only -0.06 pp. This explains the negative interactions of §IV-B: where a quantized arm already commits more often, leniency has less left to recover.

Even a single-letter answer space does not protect the delta. On ARC-Challenge we scored the same 500 items both by generation and by the log-likelihood of the four letters, which removes extraction. On Phi-3.5 and Llama-3.1 the difference between the two quantization deltas is at most 1.8 points. On

TABLE III
PRECISION × SCORING INTERACTION IN ACCURACY POINTS AGAINST `bf16`, ON 105,950 GENERATIONS. **BOLD** EXCLUDES THE NULL. †: ARC-C IS EXACTLY ZERO, SO THE INTERVAL IS THE WILSON ZERO-EVENT BOUND.

	ARC-C (gen.) (n=1,500)		BBH (n=3,300)		$\text{GSM8K}_{\text{CoT}}$ (n=3,957)	
arm	est.	95% CI	est.	95% CI	est.	95% CI
GPTQ W8A16	+0.00†	[-0.51, +0.51]	+0.06	[-0.67, +0.76]	+0.53	[-0.33, +1.42]
GPTQ W4A16	+0.00†	[-0.51, +0.51]	**+1.73**	[+0.79, +2.70]	**-5.21**	[-6.77, -3.61]
AWQ W4A16	+0.00†	[-0.51, +0.51]	**+1.55**	[+0.61, +2.52]	-0.91	[-2.50, +0.66]
bnb NF4	+0.00†	[-0.51, +0.51]	**-0.97**	[-1.91, -0.06]	**-8.44**	[-9.91, -6.95]

Qwen2.5 it is -13.2 points for GPTQ (95% CI -16.6 to -10.0) and -8.4 for AWQ, the only 2 of 6 contrasts that survive correction. Qwen's `bf16` model often answers in prose and is cut off before a letter appears, so under generation it scores 67.6% against GPTQ's 74.4%, while under log-likelihood they score 80.2% and 73.8%: the sign of the delta reverses.

V. DISCUSSION

Pin two factors and report the third. Item ordering and the process boundary are nuisance parameters, so the remedy is to pin them. Generating every compared precision inside one process removes the cross-process variation at no cost. Disabling the quantized kernels removes the ordering variation for GPTQ and AWQ at about 3.7× evaluation wall clock, and does nothing for NF4. The extraction rule is a choice about what counts as correct, and its effect varies by method and task, so it cannot be standardised away and has to be reported.

Recommend the test, not the flag. A kernel flag is specific to one engine version. The durable advice is a test: run the same items in several orderings and check that the outputs agree bitwise, for whichever kernels your backend selected. We release that probe. A protocol should also report the item submission order, whether compared precisions were generated in one process, and the generation budget with its truncation rate.

Audit your own instrument. The largest single error we found was ours: the extractor defect of §III changed two conclusions while every unit test passed, because no fixture continued past its committed answer. A census over every item found it where a fixture set could not, which is the same phenomenon this paper measures, seen from the inside.

Limitations. All measurements use one engine version, so no result should be read as a claim about other versions; the ordering result holds on two GPU architectures, and the cross-process result was measured on one, with its cause undiagnosed. Three models support no claim about a model population, and every task is a reasoning task with a regular-expression extractor. The competence decomposition rests largely on bounds, because the always-compliant stratum falls below half the paired items in 14 of 27 cells. A stock AWQ checkpoint cannot be served in `bfloat16` on this engine version at all, so published AWQ numbers are not dtype-comparable to GPTQ unless dtype is reported. Two registered hypotheses, the trace-without-answer rate and the zero-shot interaction, are not reported, because only partial numbers exist; they remain in the pre-registration.

VI. CONCLUSION

Two unreported properties of an evaluation move reported quantization degradation, and both are established by intervention or by a pre-registered contrast. Item submission order changes 4-bit accuracy while leaving `bf16` exactly unchanged, through quantized kernels that the engine's batch-invariant mode does not cover. The extraction rule interacts with precision with a sign that depends on the task and method, so it has to be reported rather than corrected for. We release the full generation corpus so that any extraction rule can be re-evaluated without a GPU.

ETHICS AND DATA STATEMENT

We evaluate existing open-weight models on public benchmarks and introduce no new model and no deployment. Every model and dataset is used within its licence: Qwen2.5-3B-Instruct is under the Qwen research licence rather than Apache-2.0, and the judge's weights (CC-BY-NC-ND-4.0) are run but never redistributed. Human labels come from two annotators following a written rubric, with the precision condition and every automated verdict withheld from the annotation file. Our results concern the *difference* between arms under a fixed protocol, not whether quantization is safe to deploy, and should not be read as licence to dismiss reported degradation. The corpus, the probe, the rubric and labels, and a log of every run (about 44 GPU-hours on single-GPU instances) are released.

REFERENCES

[1] E. Kurtic, A. Marques, S. Pandit, M. Kurtz, and D. Alistarh, ""give me bf16 or give me death"? accuracy-performance trade-offs in llm quantization," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025.

[2] H. Jo, J. Lee, J. Lee, S.-W. Lee, J. Park, and K. M. Yoo, "Finding answers in thought matters: Revisiting evaluation on large language models with reasoning," *arXiv preprint arXiv:2510.14773*, 2025.

[3] Q. Yu, Z. Zheng, S. Song, Z. Li, F. Xiong, B. Tang, and D. Chen, "xfinder: Large language models as automated evaluators for reliable evaluation," in *International Conference on Learning Representations (ICLR)*, 2025.

[4] M. Sclar, Y. Choi, Y. Tsvetkov, and A. Suhr, "Quantifying language models' sensitivity to spurious features in prompt design or: How i learned to start worrying about prompt formatting," in *International Conference on Learning Representations (ICLR)*, 2024.

[5] X. Ren, W. Zhai, T. Pu, Y. Zhu, H. Wei, and B. Zhao, "Right answer, wrong method: Shortcut hacking misleads the evaluation of llm reasoning on frontier science benchmarks," *arXiv preprint arXiv:2608.02442*, 2026, work in progress.

[6] K. Marchisio, S. Dash, H. Chen, D. Aumiller, A. Üstün, S. Hooker, and S. Ruder, "How does quantization affect multilingual llms?" in *Findings of the Association for Computational Linguistics: EMNLP 2024*, 2024.

[7] A. Dutta, S. Krishnan, N. Kwatra, and R. Ramjee, "Accuracy is not all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[8] D. Pape, J. Evertz, and L. Schönherr, "The silent hyperparameter: Quantifying the impact of inference backends on llm reproducibility," *arXiv preprint arXiv:2605.19537*, 2026.

[9] S. Masoudian, P. Shafaei, M. Swain, and M. Schedl, "What we observe as llm behavior can be a side-effect of inference backend," *arXiv preprint arXiv:2608.04714*, 2026.

[10] H. He and Thinking Machines Lab, "Defeating nondeterminism in LLM inference," Thinking Machines Lab: Connectionism, Sep. 2025. [Online]. Available: https://thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference/

[11] A. Hua, K. Tang, C. Gu, J. Gu, E. Wong, and Y. Qin, "Flaw or artifact? rethinking prompt sensitivity in evaluating llms," in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2025.

[12] X. Lian, W. Krichene, B. Huang, M. Tanaka, O. Ruwase, L. Zhang, and M. Zhang, "Quantization inflates reasoning: Token inflation as a hidden cost of low-bit reasoning models," *arXiv preprint arXiv:2606.25519*, 2026.

[13] S. Lotfi, P. Kirichenko, S. Li, and Z. Liu, "Quantized reasoning models think they need to think longer, but they do not," *arXiv preprint arXiv:2606.00206*, 2026.

[14] R. Liu, Y. Sun, M. Zhang, H. Bai, X. Yu, T. Yu, C. Yuan, and L. Hou, "Quantization hurts reasoning? an empirical study on quantized reasoning models," in *Conference on Language Modeling (COLM)*, 2025.

[15] U. Kurt, "Which quantization should i use? a unified evaluation of llama.cpp quantization on llama-3.1-8b-instruct," *arXiv preprint arXiv:2601.14277*, 2026.

[16] S. Biderman, H. Schoelkopf, L. Sutawika, L. Gao, J. Tow, B. Abbasi, A. F. Aji, P. S. Ammanamanchi, S. Black, J. Clive, A. DiPofi, J. Etxaniz, B. Fattori, J. Z. Forde, C. Foster, J. Hsu, M. Jaiswal, W. Y. Lee, H. Li, C. Lovering, N. Muennighoff, E. Pavlick, J. Phang, A. Skowron, S. Tan, X. Tang, K. A. Wang, G. I. Winata, F. Yvon, and A. Zou, "Lessons from the trenches on reproducible evaluation of language models," *arXiv preprint arXiv:2405.14782*, 2024.

[17] D. Chen, Q. Yu, P. Wang, M. Hu, W. Zhang, Z. Wang, B. Tang, F. Xiong, X. Li, C. Wang, M. Yang, and Z. Li, "xverify: Efficient answer verifier for reasoning model evaluations," *arXiv preprint arXiv:2504.10481*, 2025.